

AI Inference Engines and Instances

Strategies for Scaling LLM Infrastructure

Jonas Aaron Vander

75th Vienna Deep Learning Meetup · 42vienna · 23 September 2026

Ollama vs vLLM

Llama-3.1-8B Instruct, single A100

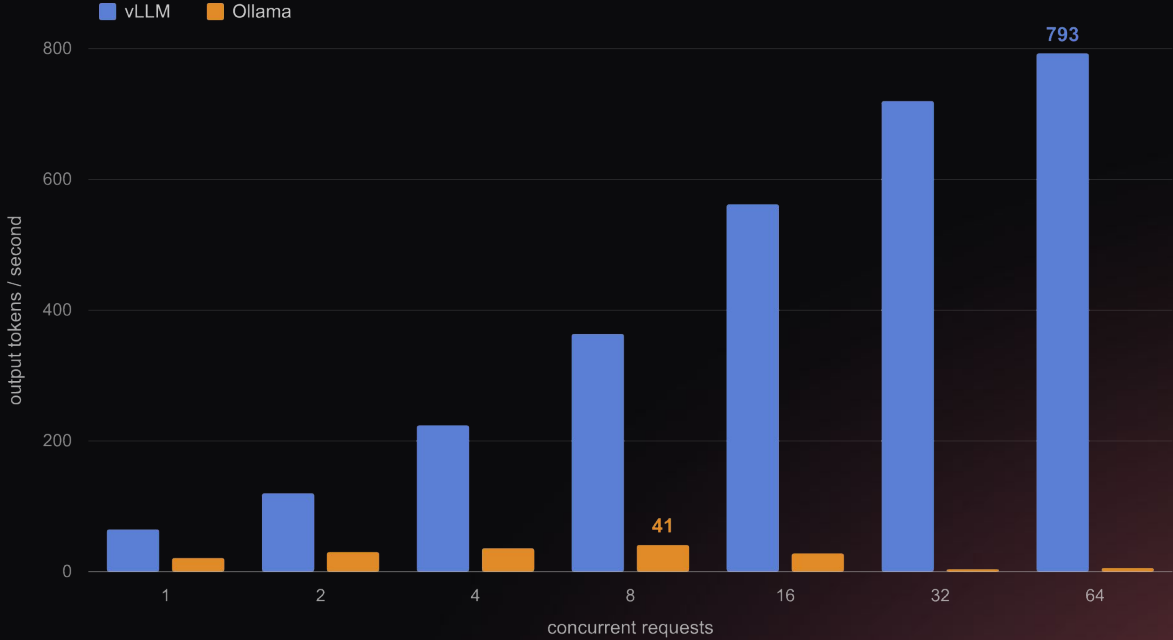
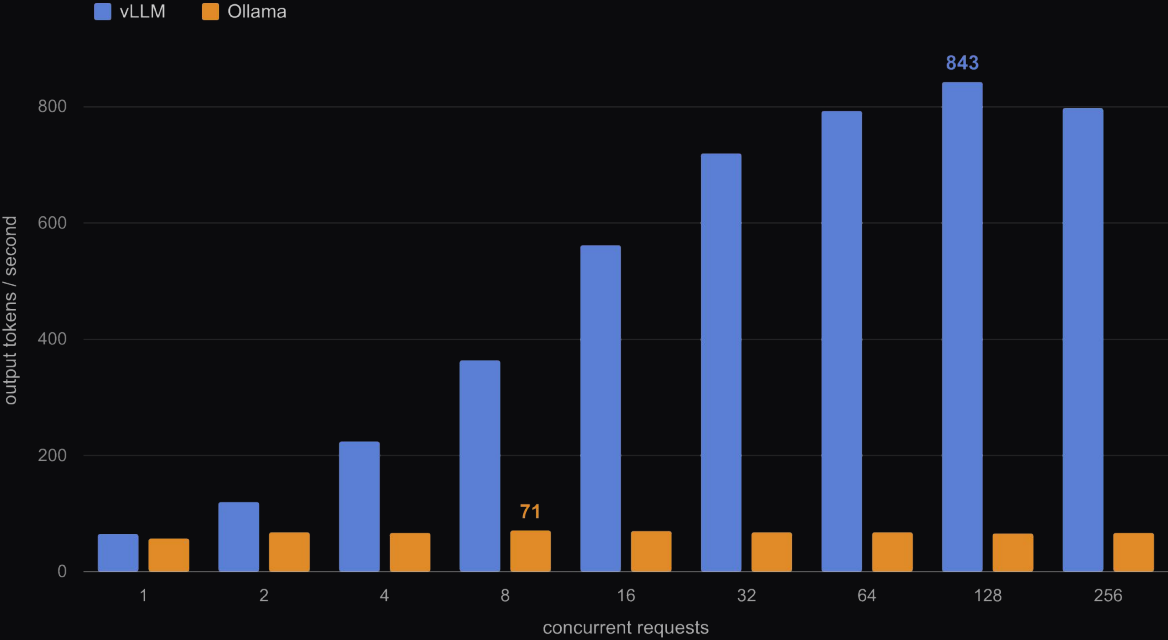
	Ollama	vLLM
Peak throughput	41 tok/s	793 tok/s
P99 latency at peak	673 ms	80 ms

19x

Same model, same hardware.

The difference is memory management and scheduling.

Ollama vs vLLM



The KV cache

Every generated token appends to a per-request cache.

It stays resident for the life of the request.

$$\text{memory} = \text{concurrency} \times \text{context length}$$

Weights are the fixed cost. This is the variable one.

Memory runs out before compute does.

Three problems

1

Memory

How do you allocate it?

2

Scheduling

How do you share it?

3

Reuse

How do you avoid
recomputing it?

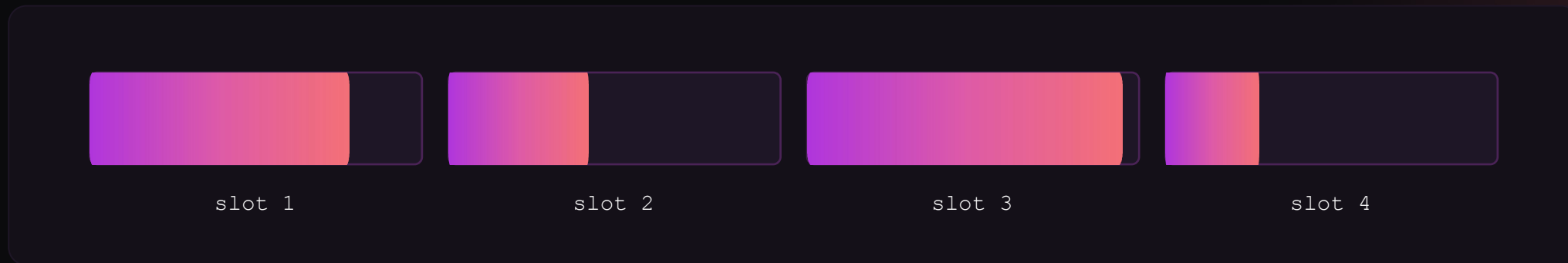
1. Memory

One shared cache, carved into slots

llama.cpp keeps a **unified KV cache**, sequences separated by masking.

Divide it across slots and each gets a fixed share.

Requests finish at unpredictable lengths. The gaps go unused.



Quantization

Llama-3.1-8B-Instruct

Format	Size	Perplexity	MMLU
F16	14.96 GiB	7.32	63.50%
Q8_0	7.95 GiB	7.33	63.43%
Q4_K_M	4.57 GiB	7.56	62.43%

Q8_0 is free. Q4_K_M costs about **one point of MMLU** for a **69% size reduction**.

The 10 GB you save becomes KV cache.

vLLM

PagedAttention: virtual memory for the KV cache

- Fixed-size **blocks**
- An **indirection table** per sequence
- A sequence's blocks can live anywhere in VRAM

The paper claims **near-zero waste** in KV cache memory, and **2-4x** the throughput of **FasterTransformer** and **Orca**.

Operating systems solved this in the 1960s.

Runs on NVIDIA, AMD and Intel. The bill is a large image, and you manage model loading yourself.

Back to the opening number

That **41 tok/s** was Ollama with its current standing default memory layout:

a hard ceiling on admissible concurrency that is set by the maximum context

llama.cpp measured their unified cache against a paged prototype (*A10G, Llama-3 8B*):

	Unified	Paged
Concurrent sequences	25-26 hard limit	~247
Throughput	496 tok/s	1256 tok/s

Ollama

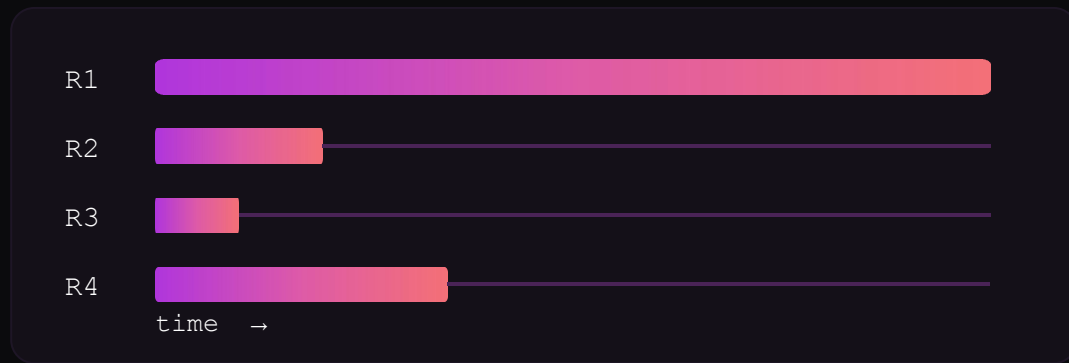
Built on llama.cpp, tuned for one user

- `curl | sh`
- `ollama run llama3.2` , and the model pulls itself
- **Layer offloading:** run models larger than your VRAM
- **Metal support:** it actually works on a Mac

All of these are single-user advantages.

2. Scheduling

Static batching



Idle slot, still held by a finished request

Batch returns when **R1** does.

Continuous batching



Next requests, pulled off the queue

Slots refill on the **next step**.

Both llama.cpp and vLLM do this.

Speculative decoding

A small model drafts ahead, the big one verifies in a single pass. The output is the same either way.

EAGLE-3 in vLLM: up to 2.5x. 1.8x on 8B / A100.

At high request rates, the 70B gains disappear.

Once the system is compute bound, the plain baseline runs faster.

3. Reuse

What does your prompt actually look like?

- The system prompt: **identical every request**
- RAG context: repeats across a session
- Few-shot examples: never change
- Agent scratchpad: grows, but the head stays frozen

Prefill is recomputation.

SGLang

RadixAttention: cached prefixes in a tree

Matched on arrival, longest prefix wins.

Shared prefix gets prefilled **once**.

vLLM matches with a hash table instead. Both reuse the prefix.

Newer than vLLM, smaller community, and the gap is version sensitive.

But this cache lives on one machine.

Structured output

This one is about correctness, not speed.

JSON schema. Regex. EBNF.

This is what function calling runs on.

XGrammar compiles the grammar to a bitmask. Under 40 μ s per token.

Both vLLM and SGLang use XGrammar by default.

The difference is **where the mask work happens**: overlapped with the GPU step, or serialised against it.

TGI

Hugging Face's own serving engine

"This repository was archived by the owner on Mar 21, 2026. It is now read-only."

Last release Dec 2025. Hugging Face now points you at vLLM and SGLang.

Where each one fits

One user, several models, sporadic use, whatever hardware you have

Ollama

Many users, one model, a GPU pool that stays busy

vLLM

same as vllm, but with workloads of typically branching messages

SGLang

Many models, many nodes

Your problem is the router, not the engine

You have a fleet

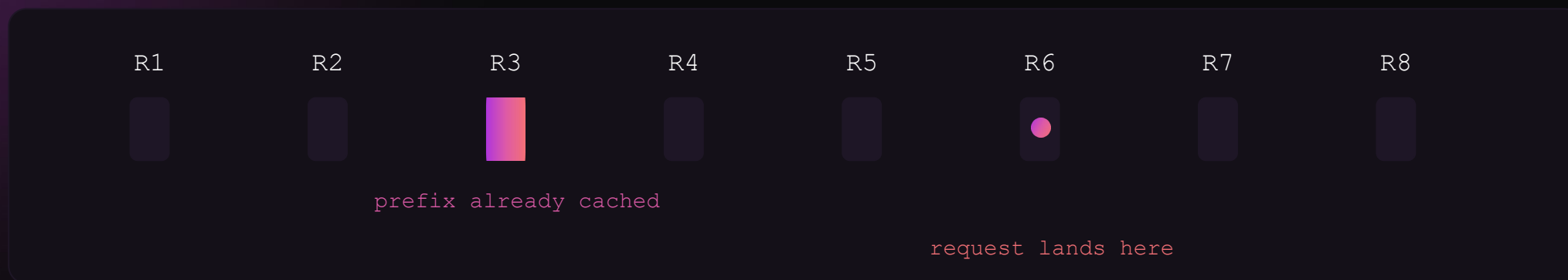
- **Ollama** at the edge, and on developer laptops
- **vLLM** for the high-traffic general model
- **SGLang** for the structured and agentic paths
- **???** for whatever replaces one of them in 18 months

They all speak an OpenAI-compatible API.

Which is why swapping one out is a day of work rather than a rewrite.

So you put a router in front

Eight replicas behind a round-robin load balancer, all healthy.



Replica 6 prefills the whole thing from scratch.

Every dashboard stays green.

It gets worse as you scale.

More replicas → lower cache locality

You scale out for latency.

Horizontal scaling is what hurts it.

The fix is known

Hash the prompt into prefix blocks.

Track which replica holds which.

Score candidates by longest match, fall back to least-loaded.

More than double the throughput of cache-blind routing.

Qwen-32B, 16xH100, simulated multi-tenant

Mean time to first token **188 ms** against **2,625 ms**. **shared-prefix workload**

Several independent implementations. Same three steps.

What it costs you, and when it does not pay

- **It only pays if your prompts actually share prefixes.** Both numbers on the last slide came from deliberately prefix-heavy workloads.
- The router becomes **stateful**. It tracks what each replica holds, and that map drifts as caches evict.
- **Cache affinity fights load balance.** Route hard for locality and you build hotspots.
- **Shared prefixes across tenants are a timing side channel.** Cache hits are measurably faster, and that leaks.
- One more component to run, monitor and debug.



The part that is still open

Those implementations are **Kubernetes-native** and assume a **homogeneous vLLM fleet**.

Your fleet has Ollama **and** vLLM **and** SGLang in it.

They block, evict and match prefixes differently.

A shared field would not mean the same thing on each engine.

**Engines come and go.
The layer above them
is the only thing you own.**

You will change engines.
Focus on the thing that lets you.

Sources

Ollama vs vLLM benchmark · Red Hat Developer

developers.redhat.com/articles/2025/08/08/ollama-vs-vllm-deep-dive-performance-benchmarking

Paged KV cache for llama.cpp · discussion #21961

github.com/ggml-org/llama.cpp/discussions/21961

Quantization evaluation on Llama-3.1-8B

[arXiv 2601.14277](https://arxiv.org/abs/2601.14277)

XGrammar

[arXiv 2411.15100](https://arxiv.org/abs/2411.15100)

PagedAttention

[arXiv 2309.06180](https://arxiv.org/abs/2309.06180)

KV-cache wins you can see · llm-d

llm-d.ai/blog/kvcache-wins-you-can-see

Prefix caching in GKE Inference Gateway · Google Cloud

TGI archive notice

github.com/huggingface/text-generation-inference